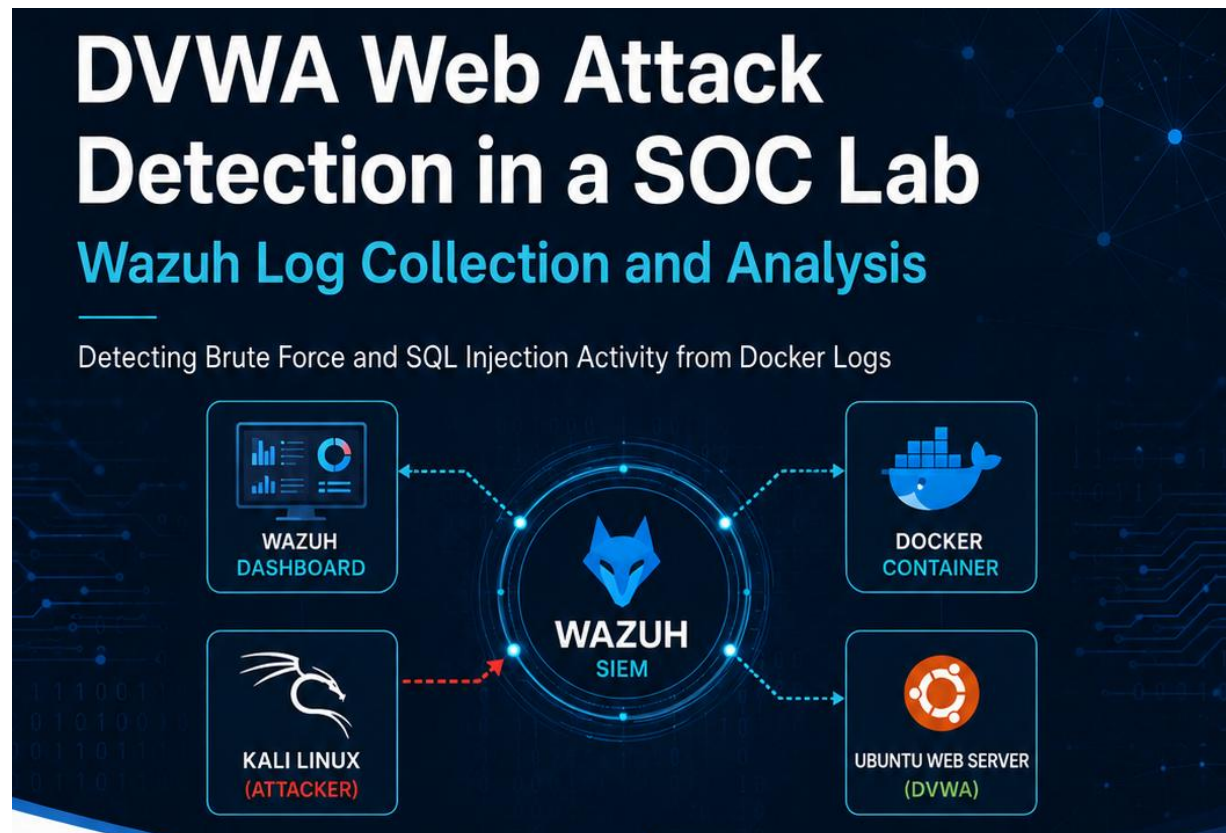




## DVWA Web Attack Detection in a SOC Lab: Wazuh Log Collection and Analysis

**RITIK PATEL**

**SOC – Threat Research Intern**

</Hacker4Help>

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

## INDEX

| Sr. No | Title                                           | Pg NO |
|--------|-------------------------------------------------|-------|
| 1      | Introduction                                    | 3     |
| 2      | Lab Architecture                                | 5     |
| 3      | Wazuh Agent<br>Installation on Ubuntu<br>Server | 6     |
| 4      | Docker Log Collection<br>Configuration          | 11    |
| 5      | Custom Wazuh<br>Detection Rules                 | 16    |
| 6      | Brute Force Attack<br>Simulation                | 21    |
| 7      | SQL Injection Attack<br>Simulation              | 25    |
| 8      | Detection and Analysis<br>Using Wazuh           | 29    |

</Hacker4Help>

9

Conclusion

33

---

## 1. Introduction

---

In the previous part of this SOC lab project, an Ubuntu web server was integrated into the existing SOC environment, Docker was installed, and DVWA was deployed as a vulnerable web application.

This second part focuses on web attack detection using Wazuh SIEM. Since DVWA was running inside Docker, Docker container logs were collected through the Wazuh agent and forwarded to the Wazuh dashboard for analysis.

In this phase, Kali Linux was used to perform controlled brute force and SQL injection attacks against DVWA. Custom Wazuh local rules were created to detect this activity and generate alerts in the dashboard.

The objective of this report is to understand how containerized web application activity appears in logs and how Wazuh can be used to detect and investigate web attacks from a SOC analyst perspective.

The objectives of this phase are:

- Configure Wazuh agent on the Ubuntu web server
- Collect Docker container logs from DVWA
- Create local Wazuh rules for web attack detection
- Simulate brute force and SQL injection attacks from Kali Linux
- Detect and analyze DVWA attack activity in Wazuh SIEM

---

## 2. Lab Environment

---

### 2.1 Lab Architecture

The lab environment used for this phase is the same SOC lab setup that was expanded in the previous report. The Ubuntu web server was already installed, Docker was configured, and DVWA was deployed as a containerized vulnerable web application.

In this phase, the Ubuntu web server was connected to Wazuh SIEM for log collection and attack detection. Kali Linux was used as the attacker machine to perform controlled brute force and SQL injection attacks against DVWA.

#### **Components of the Lab:**

1. Attacker Machine: Kali Linux used to perform web application attacks
2. Web Server: Ubuntu Server hosting DVWA using Docker
3. Vulnerable Web Application: DVWA used as the target application
4. Container Platform: Docker used to run DVWA and its database in isolated containers
5. SIEM Platform: Wazuh used to collect logs, apply detection rules, and analyze alerts
6. Network Configuration: All machines were connected within the same VirtualBox NAT network

### 2.2 Environment purpose

The purpose of this environment was to monitor web application attack activity from a SOC analyst perspective.

</Hacker4Help>

Since DVWA was running inside Docker, the attack activity was recorded inside Docker container logs. These logs were collected by the Wazuh agent installed on the Ubuntu web server and forwarded to the Wazuh dashboard.

This setup helped demonstrate how containerized web application activity can be collected, detected, and analyzed using Wazuh SIEM.

## 2.3 Scope of the Environment

This phase focuses on web application attack detection using Wazuh. The simulation includes:

- Wazuh agent installation on the Ubuntu web server
- Docker container log collection configuration
- Docker listener configuration
- Custom Wazuh local rule creation
- Brute force attack simulation against DVWA
- SQL injection attack simulation against DVWA
- Local Docker log verification
- Detection and analysis of DVWA alerts in Wazuh SIEM

The goal is to understand how DVWA attack activity appears in Docker logs and how Wazuh can be configured to detect and investigate these events in a SOC environment.

---

## 3. Wazuh Agent Installation on Ubuntu Web Server

---

### 3.1 Overview

After DVWA was deployed on the Ubuntu web server, the next step was to connect the server with Wazuh SIEM. The Wazuh agent was installed on the Ubuntu web server so that logs from the server could be collected and forwarded to the Wazuh dashboard.

This was required because the Ubuntu web server was the system hosting DVWA, and any web attack activity against DVWA needed to be monitored from this machine.

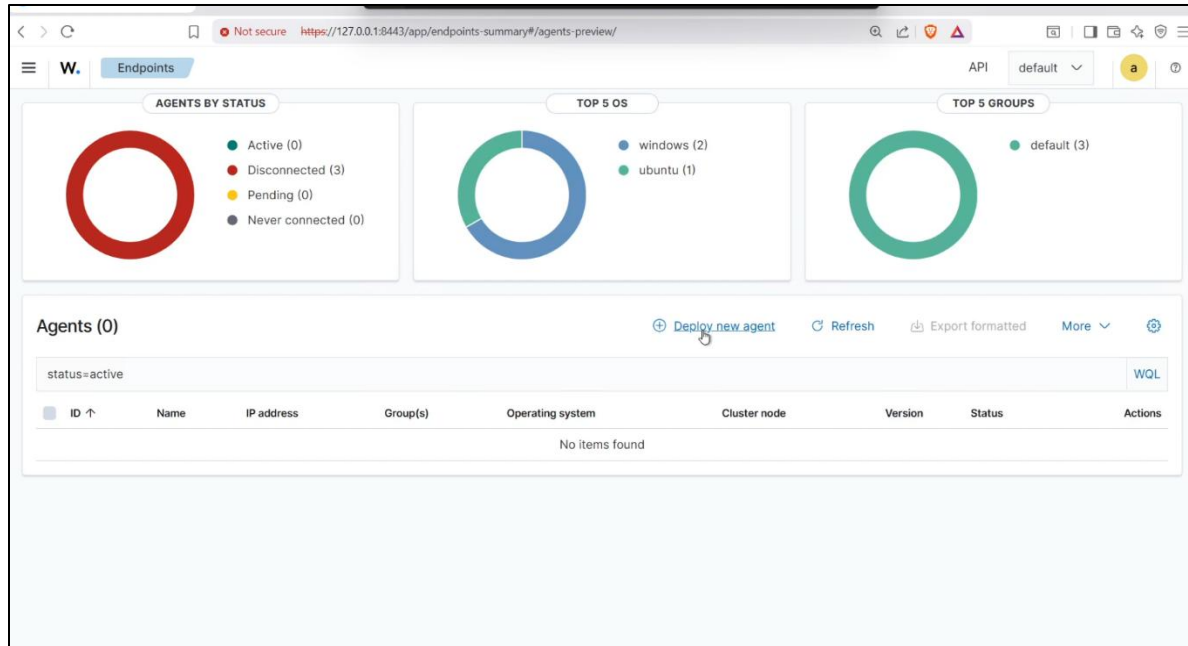
### 3.2 Deploying New Agent from Wazuh Dashboard

The Wazuh dashboard was used to generate the agent installation command for the Ubuntu web server.

Steps Performed:

1. Logged in to the Wazuh dashboard
2. Opened the Agents section
3. Clicked Deploy new agent
4. Selected Linux as the operating system
5. Selected DEB amd64 package
6. Entered the Wazuh server address
7. Set the agent name as: `web-server`
8. Copied the installation commands generated by Wazuh

</Hacker4Help>



The screenshot shows the 'Deploy new agent' page. It has a back arrow and the title 'Deploy new agent'. The first section is 'Select the package to download and install on your system:' with three tabs: 'LINUX', 'WINDOWS', and 'macOS'. The 'LINUX' tab is selected, showing options for 'RPM amd64', 'RPM aarch64', 'DEB amd64' (selected), and 'DEB aarch64'. The 'WINDOWS' tab shows 'MSI 32/64 bits'. The 'macOS' tab shows 'Intel' and 'Apple silicon'. Below the tabs is a link to 'documentation'. The second section is 'Server address:' with a description and a text input field containing '172.16.1.5'. There is also a 'Remember server address' checkbox which is checked.

Deploy new agent

Select the package to download and install on your system:

**LINUX**

- ☐ RPM amd64
- ☐ RPM aarch64
- ☒ DEB amd64
- ☐ DEB aarch64

**WINDOWS**

- ☐ MSI 32/64 bits

**macOS**

- ☐ Intel
- ☐ Apple silicon

For additional systems and architectures, please check our [documentation](#).

Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

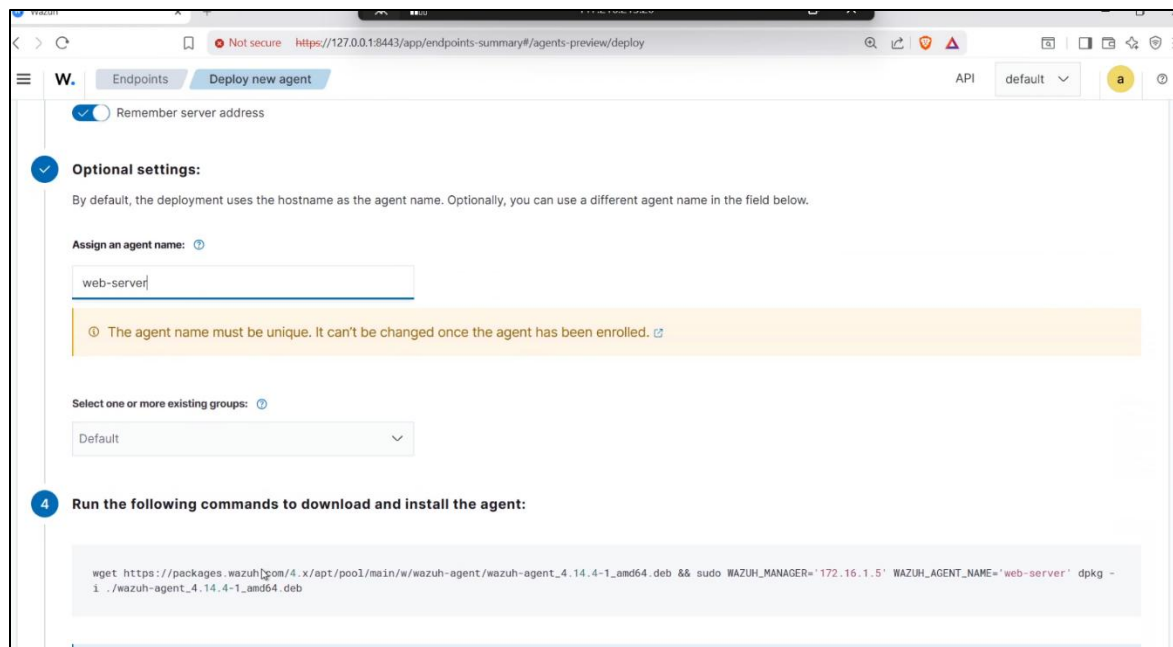
Assign a server address

172.16.1.5

☒ Remember server address

Ritik Patel  
SOC - Threat Research intern at </Hacker4Help>





### 3.3 Installing Wazuh Agent on Ubuntu Server

The generated Wazuh agent installation command was executed on the Ubuntu web server.

After installation, the Wazuh agent service was enabled and started.

#### Commands Used:

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable wazuh-agent
```

```
sudo systemctl start wazuh-agent
```

```
ritik@web-server:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.4-1_amd64.deb && sudo WAZUH_MANAGER='172.16.1.5' WAZUH_AGENT_NAME='web-server' dpkg -i ./wazuh-agent_4.14.4-1_amd64.deb
--2026-05-17 19:15:57-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.4-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 108.157.238.17, 108.157.238.38, 108.157.238.50, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|108.157.238.17|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13220908 (13M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.14.4-1_amd64.deb'

wazuh-agent_4.14.4-1_amd64.deb      100%[=====] 12.61M  5.07MB/s   in 2.5s

2026-05-17 19:16:01 (5.07 MB/s) - 'wazuh-agent_4.14.4-1_amd64.deb' saved [13220908/13220908]

[sudo] password for ritik:
Selecting previously unselected package wazuh-agent.
(Reading database ... 88689 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.14.4-1_amd64.deb ...
Unpacking wazuh-agent (4.14.4-1) ...
Setting up wazuh-agent (4.14.4-1) ...
```

```
ritik@web-server:~$ sudo systemctl daemon-reload
ritik@web-server:~$ sudo systemctl enable wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service → /usr/lib/systemd/system/wazuh-agent.service.
ritik@web-server:~$ sudo systemctl start wazuh-agent
ritik@web-server:~$
```

### 3.4 Verifying Wazuh Agent Status Locally

After installation, the Wazuh agent status was checked on the Ubuntu web server.

#### Command Used:

```
sudo systemctl status wazuh-agent
```

### 3.5 Verifying Agent in Wazuh Dashboard

After starting the agent, the Wazuh dashboard was checked to confirm that the Ubuntu web server was connected.

#### Observation:

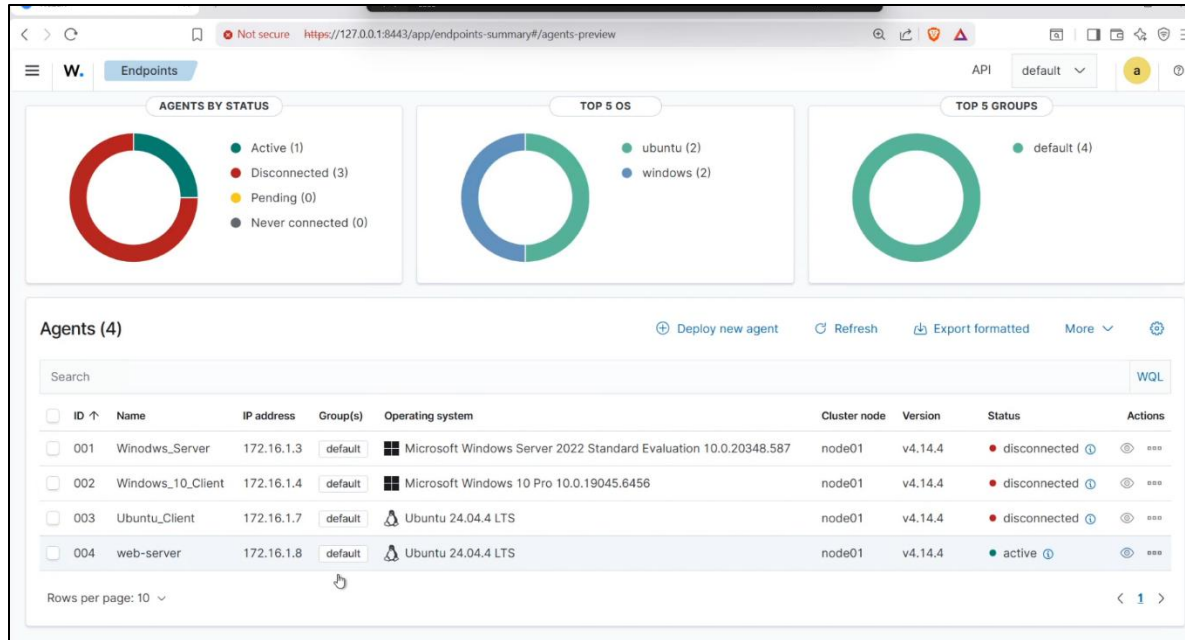
The Ubuntu web server appeared in the Wazuh dashboard with the agent name:

```
web-server
```

The agent status was shown as:

```
Active
```

This confirmed that the Ubuntu web server was successfully connected to Wazuh SIEM.



### 3.6 Outcome

After completing the installation, the Ubuntu web server was successfully connected to Wazuh SIEM.

---

## 4. Docker Log Collection Configuration

---

### 4.1 Overview

After the Wazuh agent was installed on the Ubuntu web server, Docker log collection was configured. This step was required because DVWA was running inside Docker containers, so the web application activity was stored in Docker container logs instead of only normal Linux log files.

The main Docker log path used in this project was:

```
/var/lib/docker/containers/*/*-json.log
```

This path allows Wazuh to collect JSON log files from Docker containers running on the Ubuntu web server.

### 4.2 Why Docker Log Collection Was Required

By default, Wazuh monitors common Linux log sources such as authentication logs and system logs. However, DVWA was deployed inside Docker, so web requests generated by Kali Linux were recorded inside Docker container logs.

For example, activity such as visiting DVWA pages, accessing the brute force module, and testing SQL injection generated logs inside the DVWA Docker container.

Because of this, Docker logs had to be added manually to the Wazuh agent configuration.

### 4.3 Updating Wazuh Agent Configuration

The Wazuh agent configuration file was opened on the Ubuntu web server.

#### Command Used:

```
sudo nano /var/ossec/etc/ossec.conf
```

The following configuration was added before the closing </ossec\_config> tag:

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

## </Hacker4Help>

```
<localfile>
  <log_format>json</log_format>
  <location>/var/lib/docker/containers/*/*-json.log</location>
</localfile>

<wodle name="docker-listener">
  <disabled>no</disabled>
</wodle>
```

### Explanation:

- localfile tells Wazuh to monitor a specific log file path
- log\_format json tells Wazuh that Docker logs are stored in JSON format
- /var/lib/docker/containers/\*/\*-json.log collects logs from all Docker containers
- docker-listener enables Docker event monitoring

```
sudo: nano: command not found
ritik@web-server:~$ sudo nano /var/ossec/etc/ossec.conf
```

```
<ossec_config>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/dpkg.log</location>
  </localfile>

  <localfile>
    <log_format>json</log_format>
    <location>/var/lib/docker/containers/*/*-json.log</location>
  </localfile>

  <wodle name="docker-listener">
    <disabled>no</disabled>
  </wodle>
</ossec_config>
```

## 4.4 Restarting Wazuh Agent

After updating the configuration file, the Wazuh agent was restarted to apply the changes.

### Command Used:

```
sudo systemctl restart wazuh-agent
```

A terminal window screenshot showing a command being executed. The prompt is 'ritik@web-server:~\$' and the command is 'sudo systemctl restart wazuh-agent'. The output is partially visible as 'ailed to restart 'wazuh-agent.service': Unit 'wazuh-agent.servi'.

## 4.5 Checking Wazuh Agent Logs for Errors

After restarting the Wazuh agent, the local Wazuh agent log was checked to make sure there were no configuration errors.

### Command Used:

```
sudo tail -n 20 /var/ossec/logs/ossec.log
```

### Explanation:

This command shows the latest Wazuh agent logs. It helps confirm whether the configuration file was loaded successfully or whether there were any XML or log collection errors.

### Observation:

No major configuration errors were found, which confirmed that the Docker log collection configuration was accepted by the Wazuh agent.

```
ritik@web-server:~$ sudo tail -n 20 /var/ossec/logs/ossec.log
2026/05/17 20:09:45 wazuh-modulesd: INFO: Started (pid: 6749).
2026/05/17 20:09:45 wazuh-modulesd:agent-upgrade: INFO: (8153): Module Agent Upgrade started.
2026/05/17 20:09:45 wazuh-modulesd:ciscat: INFO: Module disabled. Exiting...
2026/05/17 20:09:45 wazuh-modulesd:osquery: INFO: Module disabled. Exiting...
2026/05/17 20:09:45 sca: INFO: Module started.
2026/05/17 20:09:45 sca: INFO: Loaded policy '/var/ossec/ruleset/sca/cis_ubuntu24-04.yml'
2026/05/17 20:09:45 wazuh-modulesd:control: INFO: Starting control thread.
2026/05/17 20:09:45 wazuh-modulesd:docker-listener: INFO: Module docker-listener started.
2026/05/17 20:09:45 sca: INFO: Starting Security Configuration Assessment scan.
2026/05/17 20:09:46 wazuh-modulesd:syscollector: INFO: Module started.
2026/05/17 20:09:46 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/05/17 20:09:46 sca: INFO: Starting evaluation of policy: '/var/ossec/ruleset/sca/cis_ubuntu24-04.yml'
2026/05/17 20:09:47 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/05/17 20:09:47 wazuh-logcollector: INFO: (9203): Monitoring journal entries.
2026/05/17 20:09:51 wazuh-syscheckd: INFO: (6009): File integrity monitoring scan ended.
2026/05/17 20:09:51 wazuh-syscheckd: INFO: FIM sync module started.
2026/05/17 20:09:51 sca: INFO: Evaluation finished for policy '/var/ossec/ruleset/sca/cis_ubuntu24-04.yml'
2026/05/17 20:09:52 sca: INFO: Security Configuration Assessment scan finished. Duration: 7 seconds.
2026/05/17 20:10:26 rootcheck: INFO: Ending rootcheck scan.
2026/05/17 20:10:45 wazuh-modulesd:docker-listener: INFO: Starting to listening Docker events.
ritik@web-server:~$ _
```

## 4.6 Verifying DVWA Docker Logs Locally

Before checking the Wazuh dashboard, DVWA logs were verified locally on the Ubuntu web server.

### Command Used:

```
sudo docker logs dvwa-dvwa-1 --tail=10
```

### Explanation:

This command displays recent logs from the DVWA web application container.

### Observation:

DVWA logs showed web requests generated from Kali Linux. The logs included DVWA paths such as:

```
/vulnerabilities/brute/
```

```
/vulnerabilities/sql/
```

This confirmed that DVWA web activity was being recorded inside Docker logs.

```
ritik@web-server:~$ sudo docker logs dvwa-dvwa-1 --tail 10
Error response from daemon: No such container: dvwa-dvwa-1
ritik@web-server:~$ sudo docker logs dvwa-dvwa-1 --tail 10
172.16.1.6 - - [17/May/2026:19:03:56 +0000] "GET /vulnerabilities/brute/ HTTP/1.1" 200 1863 "http://172.16.1.8:4280/vulnerabilities/sqli/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:17:41 +0000] "POST /vulnerabilities/brute/ HTTP/1.1" 200 2148 "http://172.16.1.8:4280/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:17:47 +0000] "GET /dvwa/js/add_event_listeners.js HTTP/1.1" 200 618 "http://172.16.1.8:4280/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:18:12 +0000] "POST /vulnerabilities/brute/ HTTP/1.1" 200 2147 "http://172.16.1.8:4280/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:18:29 +0000] "POST /vulnerabilities/brute/ HTTP/1.1" 200 2145 "http://172.16.1.8:4280/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:18:38 +0000] "POST /vulnerabilities/brute/ HTTP/1.1" 200 2145 "http://172.16.1.8:4280/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:19:24 +0000] "GET /vulnerabilities/exec/ HTTP/1.1" 200 1904 "http://172.16.1.8:4280/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:19:26 +0000] "GET /vulnerabilities/brute/ HTTP/1.1" 200 1862 "http://172.16.1.8:4280/vulnerabilities/exec/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:19:34 +0000] "POST /vulnerabilities/brute/ HTTP/1.1" 200 2149 "http://172.16.1.8:4280/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
172.16.1.6 - - [17/May/2026:20:19:48 +0000] "POST /vulnerabilities/brute/ HTTP/1.1" 200 2149 "http://172.16.1.8:4280/vulnerabilities/brute/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
```

## 4.7 Outcome

After completing this configuration, the Wazuh agent was able to collect Docker container logs from the Ubuntu web server.



---

## 5. Custom Wazuh Detection Rules

---

### 5.1 Overview

After Docker log collection was configured, custom Wazuh local rules were created to detect DVWA-related web attack activity.

Although Docker logs were being collected by the Wazuh agent, Wazuh needed specific rules to identify DVWA brute force and SQL injection activity as security alerts. These rules were created on the Wazuh server.

### 5.2 Why Local Rules Were Required

DVWA was running inside Docker, so the web activity was collected as Docker JSON logs. By default, Wazuh may collect these logs, but it may not automatically generate clear alerts for DVWA-specific paths such as:

`/vulnerabilities/brute/`

`/vulnerabilities/sqli/`

Because of this, local rules were created to match these DVWA paths and generate alerts in the Wazuh dashboard.

### 5.3 Editing Local Rules File

Steps performed:

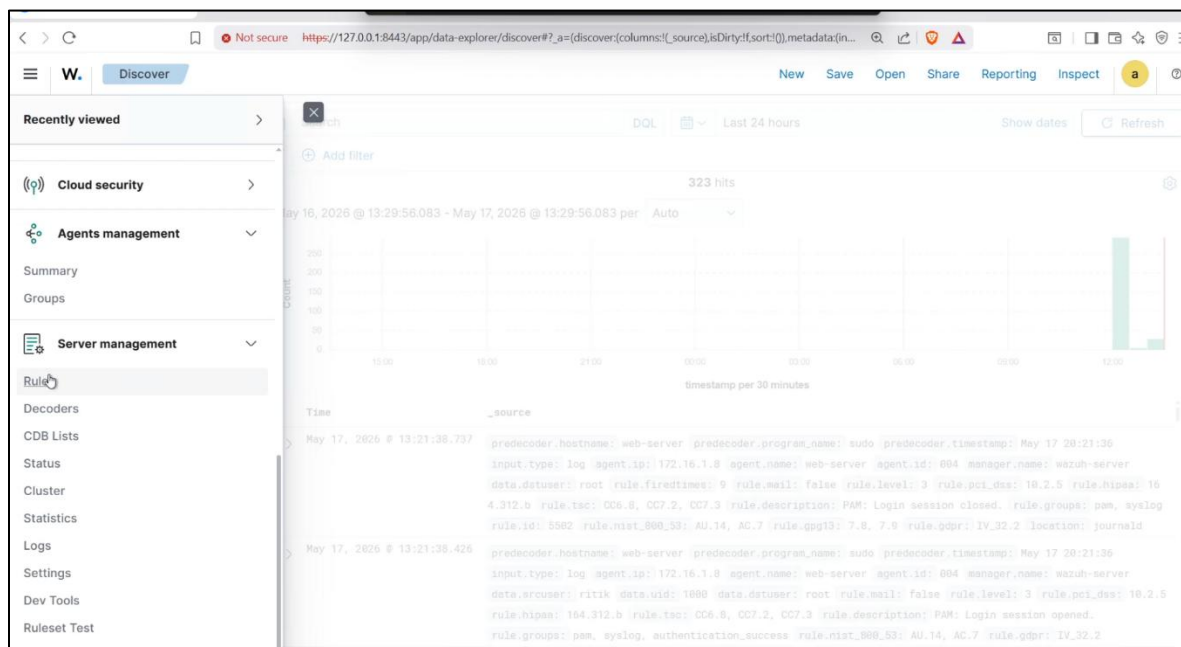
1. Go to Wazuh dashboard
2. Navigate to Server Management -> Rules
3. Then go to Custom rules
4. Click on local\_rules.xml
5. Add the following rules at the end, click on Save button, and click on Reload

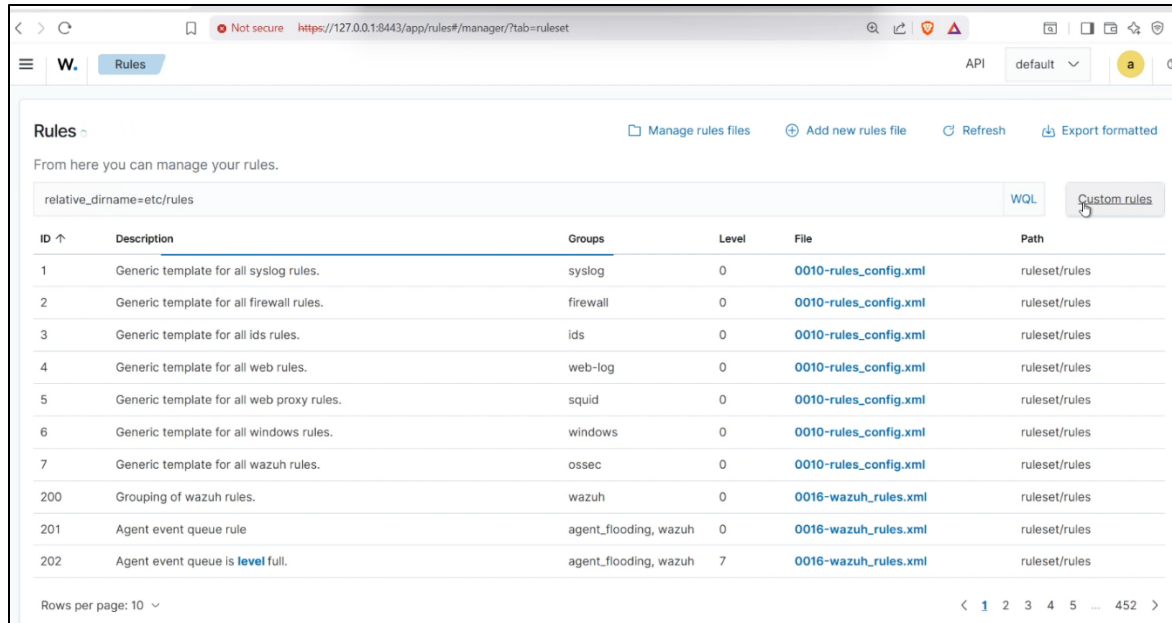
```
<group name="dvwa,web,docker,">
  <rule id="100500" level="5">
    <match>/vulnerabilities/brute/</match>
    <description>DVWA brute force page activity detected in Docker web logs</description>
  </rule>

  <rule id="100501" level="5">
    <match>/vulnerabilities/sqli/</match>
    <description>DVWA SQL injection page activity detected in Docker web logs</description>
  </rule>
</group>
```

## Explanation:

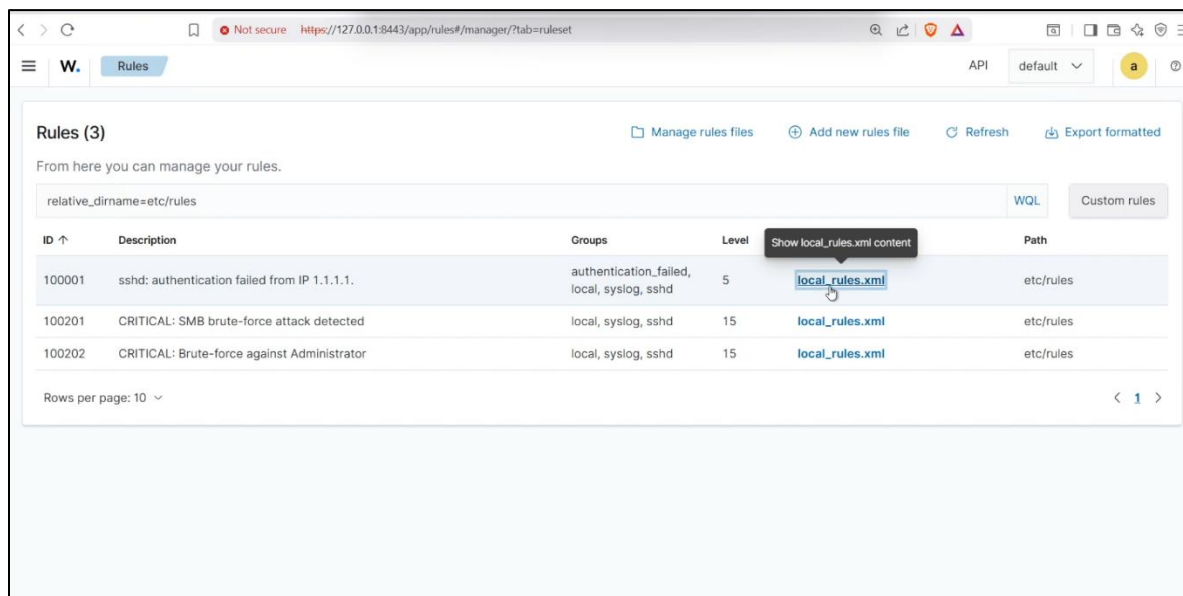
- Rule 100500 detects activity on the DVWA brute force page
- Rule 100501 detects activity on the DVWA SQL injection page
- The match field searches for specific DVWA paths inside Docker logs
- The description field defines the alert message shown in Wazuh
- The rule level was set to 5 to generate visible alerts for lab testing





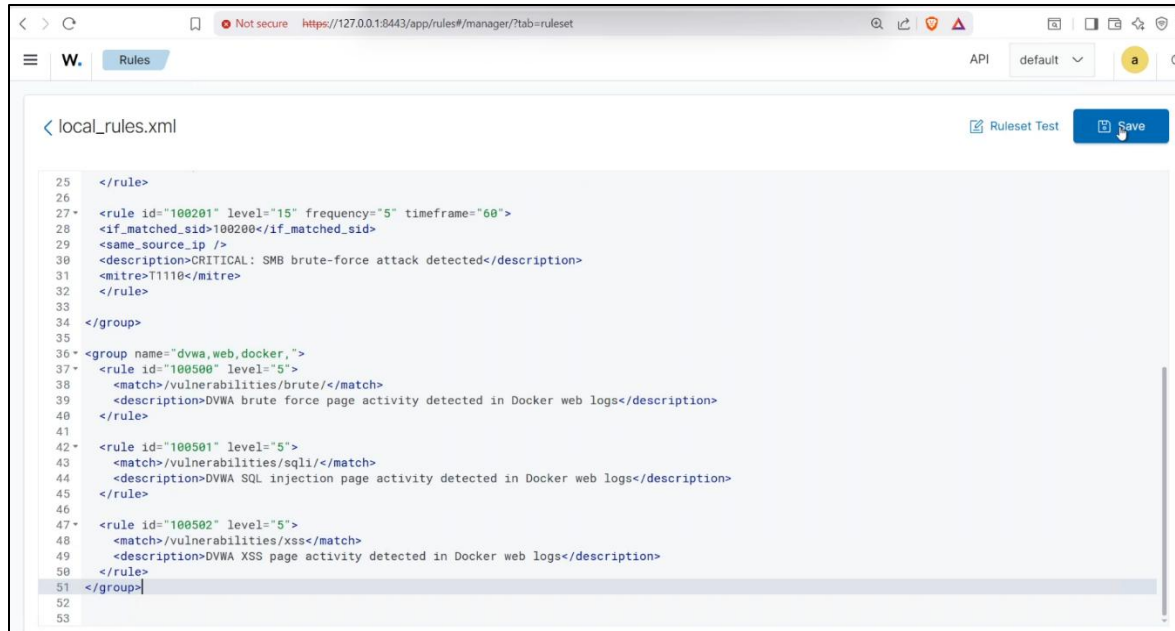
The screenshot shows the Wazuh Rules Manager interface. The browser address bar displays `https://127.0.0.1:8443/app/rules#/manager/?tab=ruleset`. The page title is "Rules". Below the title, there are links for "Manage rules files", "Add new rules file", "Refresh", and "Export formatted". A search bar contains the text "relative\_dirname=etc/rules". To the right of the search bar are buttons for "WQL" and "Custom rules". The main content is a table of rules with the following columns: ID, Description, Groups, Level, File, and Path. The table lists 202 rules, with the first 7 being generic templates and the last 3 being specific rules. The "File" column shows the path to the rule file, and the "Path" column shows the path to the rule set.

| ID  | Description                               | Groups                | Level | File                  | Path          |
|-----|-------------------------------------------|-----------------------|-------|-----------------------|---------------|
| 1   | Generic template for all syslog rules.    | syslog                | 0     | 0010-rules_config.xml | ruleset/rules |
| 2   | Generic template for all firewall rules.  | firewall              | 0     | 0010-rules_config.xml | ruleset/rules |
| 3   | Generic template for all ids rules.       | ids                   | 0     | 0010-rules_config.xml | ruleset/rules |
| 4   | Generic template for all web rules.       | web-log               | 0     | 0010-rules_config.xml | ruleset/rules |
| 5   | Generic template for all web proxy rules. | squid                 | 0     | 0010-rules_config.xml | ruleset/rules |
| 6   | Generic template for all windows rules.   | windows               | 0     | 0010-rules_config.xml | ruleset/rules |
| 7   | Generic template for all wazuh rules.     | ossec                 | 0     | 0010-rules_config.xml | ruleset/rules |
| 200 | Grouping of wazuh rules.                  | wazuh                 | 0     | 0016-wazuh_rules.xml  | ruleset/rules |
| 201 | Agent event queue rule                    | agent_flooding, wazuh | 0     | 0016-wazuh_rules.xml  | ruleset/rules |
| 202 | Agent event queue is level full.          | agent_flooding, wazuh | 7     | 0016-wazuh_rules.xml  | ruleset/rules |



The screenshot shows the Wazuh Rules Manager interface. The browser address bar displays `https://127.0.0.1:8443/app/rules#/manager/?tab=ruleset`. The page title is "Rules (3)". Below the title, there are links for "Manage rules files", "Add new rules file", "Refresh", and "Export formatted". A search bar contains the text "relative\_dirname=etc/rules". To the right of the search bar are buttons for "WQL" and "Custom rules". The main content is a table of rules with the following columns: ID, Description, Groups, Level, File, and Path. The table lists 3 rules, with the first being a specific rule and the last two being generic templates. The "File" column shows the path to the rule file, and the "Path" column shows the path to the rule set. A tooltip is visible over the "local\_rules.xml" link in the "File" column of the first rule, displaying the text "Show local\_rules.xml content".

| ID     | Description                                  | Groups                                     | Level | File            | Path      |
|--------|----------------------------------------------|--------------------------------------------|-------|-----------------|-----------|
| 100001 | sshd: authentication failed from IP 1.1.1.1. | authentication_failed, local, syslog, sshd | 5     | local_rules.xml | etc/rules |
| 100201 | CRITICAL: SMB brute-force attack detected    | local, syslog, sshd                        | 15    | local_rules.xml | etc/rules |
| 100202 | CRITICAL: Brute-force against Administrator  | local, syslog, sshd                        | 15    | local_rules.xml | etc/rules |



## 5.4 Restarting Wazuh Manager

After adding the local rules, the Wazuh manager service was restarted to apply the changes.

### Command Used:

```
sudo systemctl restart wazuh-manager
```

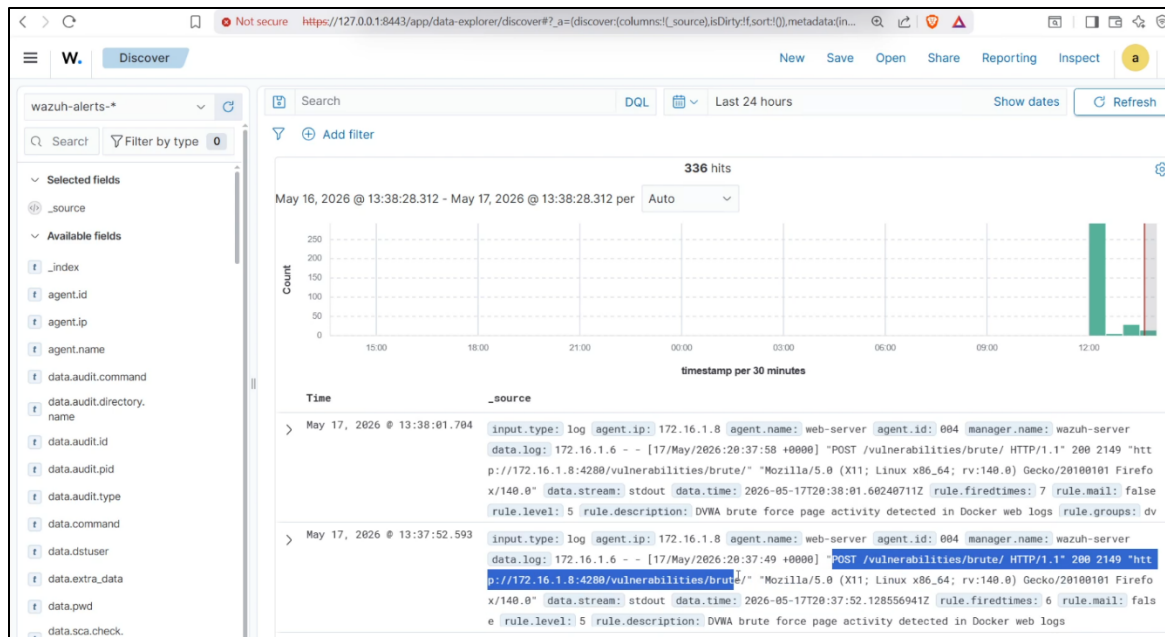
The Wazuh manager status was checked using:

```
sudo systemctl status wazuh-manager
```

```
[wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-manager
■ wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-05-17 20:34:47 UTC; 20s ago
     Process: 5713 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/
    Tasks: 231 (limit: 9478)
      Memory: 878.9M
         CPU: 48.974s
    CGroup: /system.slice/wazuh-manager.service
           └─5777 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─5778 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─5779 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
```

## 5.5 Rule Verification

After the custom rules were added, DVWA activity was generated from Kali Linux and checked in the Wazuh dashboard.



## 5.6 Outcome

Custom Wazuh local rules were successfully created and applied.

---

## 6. Brute Force Attack Simulation

---

### 6.1 Overview

After configuring Wazuh log collection and custom detection rules, a controlled brute force attack was performed against the DVWA brute force module. Kali Linux was used as the attacker machine, and DVWA running on the Ubuntu web server was used as the target application.

The purpose of this simulation was to generate repeated login activity against DVWA and verify that the activity could be captured in Docker logs and detected in the Wazuh dashboard.

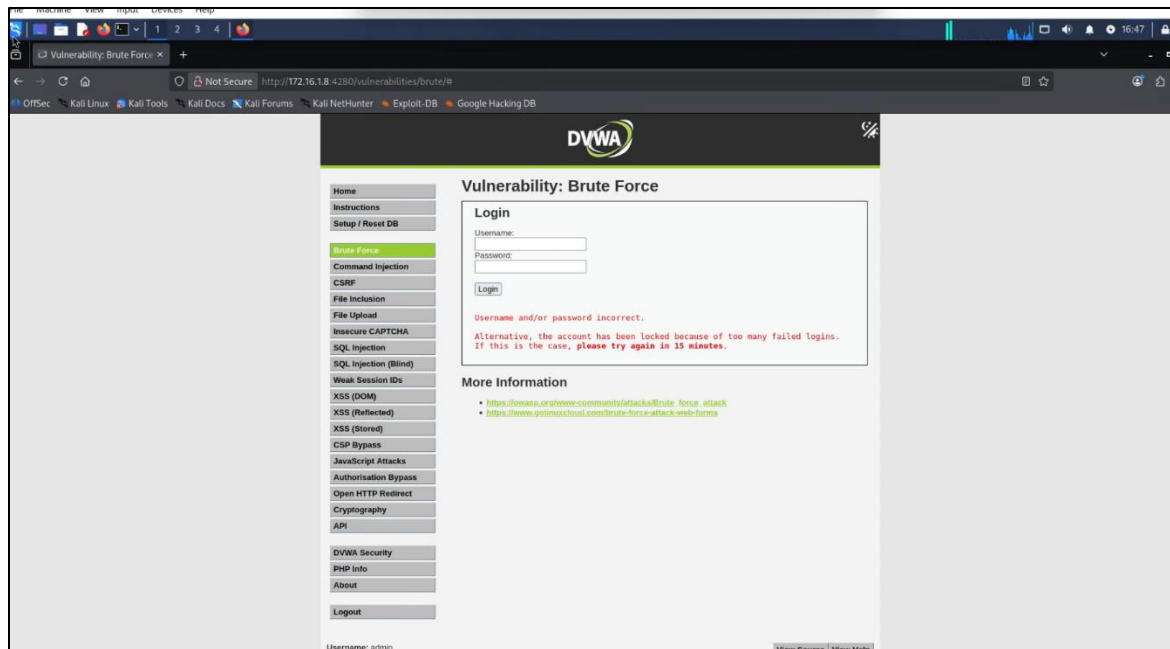
During brute force attack simulation, the security level of DVWA was set to impossible.

### 6.2 Accessing DVWA Brute Force Module

The DVWA brute force module was accessed from Kali Linux using the browser.

**URL Used:**

<http://172.16.1.8:4280/vulnerabilities/brute/>



## 6.3 Hydra-Based Brute Force Attempt

Hydra was also used from Kali Linux to perform automated login attempts against the DVWA brute force page.

### Command Used:

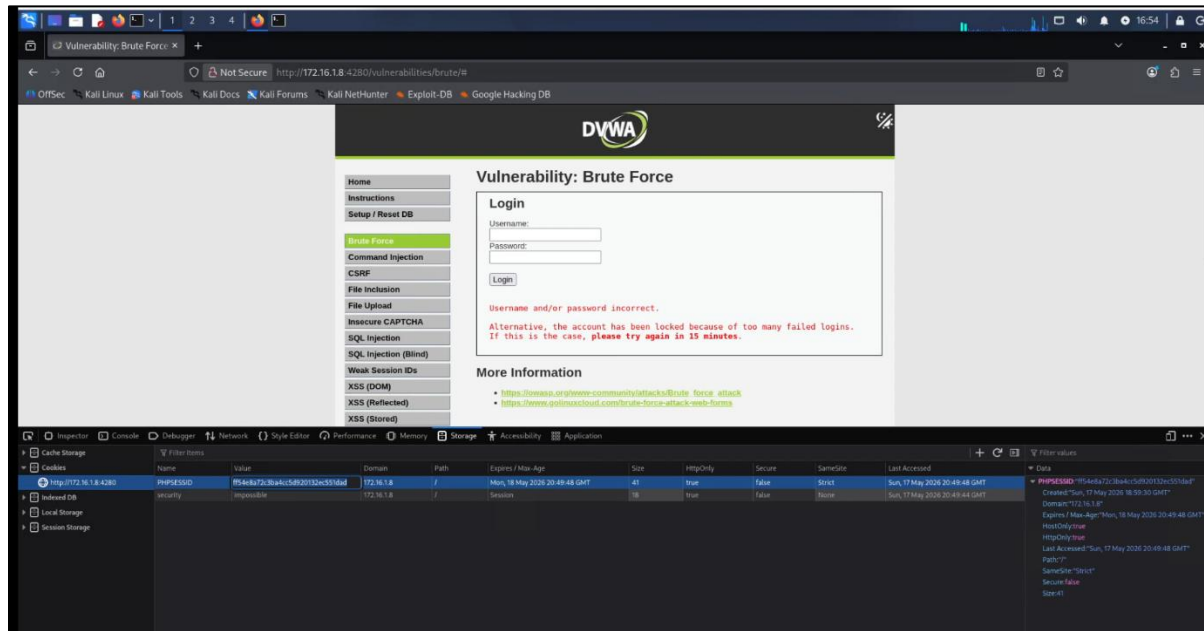
```
hydra -l admin -P password_list.txt -s 4280 172.16.1.8 http-get-form  
'/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=  
Cookie\: security=low; PHPSESSID=SESSION_ID:F=Username and/or password  
incorrect.' -V
```

### Explanation:

- hydra was used to automate the brute force attempt
- -l admin specifies the username
- -P password\_list.txt specifies the password list
- 172.16.1.8 is the target Ubuntu web server
- -s 4280 specifies the DVWA web port
- http-get-form targets the DVWA login form
- PHPSESSID was used to maintain the active DVWA session

## </Hacker4Help>

- -V displays each login attempt



```
root@kali:~/home/kali/Desktop# hydra -l admin -P password_list.txt -s 4280 172.16.1.8 http-get-form '/vulnerabilities/brute/?username=admin&password=PASSWORD' --cookie 'PHPSESSID=ff54e8a72c3ba4cc5d920132ec551dad;F=Username and/or password incorrect.' -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-17 17:05:21
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[DATA] max 10 tasks per 1 server, overall 10 tasks, 18 login tries (12/2:10), -1 try per task
[DATA] attacking http-get-form://172.16.1.8:4280/vulnerabilities/brute/?username=USER&password=PASSWORD: security=low; PHPSESSID=ff54e8a72c3ba4cc5d920132ec551dad;F=Username and/or password incorrect.
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "AdminHacker6453" - 1 of 10 [child 0] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "Admin2183abc" - 2 of 10 [child 1] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "User@it.kali" - 3 of 10 [child 2] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "wrieLess_Root" - 4 of 10 [child 3] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "admin_admin_463" - 5 of 10 [child 4] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "99221322$admin" - 6 of 10 [child 5] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "lgj1f892k" - 7 of 10 [child 6] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "hdieh66258fddw" - 8 of 10 [child 7] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "Austosh$haiK$" - 9 of 10 [child 8] (0/0)
[ATTEMPT] target 172.16.1.8 - login "admin" - pass "8KSPpp9?" - 10 of 10 [child 9] (0/0)
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-17 17:05:21
```

## 6.4 Verifying Brute Force Logs Locally

After performing the brute force attempts, Docker logs were checked locally on the Ubuntu web server.

### Command Used:

```
sudo docker logs dvwa-dvwa-1 --tail=10
```

### Observation:

The Docker logs showed repeated requests to the DVWA brute force module.

Example log indicator:

```
GET /vulnerabilities/brute/?username=admin
```

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>



## </Hacker4Help>

This confirmed that the brute force activity was recorded inside the DVWA Docker container logs.

```
ritik@web-server:~$ sudo docker logs dvwa-dvwa-1 --tail 10
[sudo] password for ritik:
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=X23AdminHacker6453&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=IsjifX26524&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=Admin02103abc&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=User0Rit_Kali&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=Wrieless_Root&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=admin_admin_463&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=IhdiehX26625Rfddw&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=99221322$admin&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=Aushtosh$bbaix25$&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
172.16.1.6 - - [17/May/2026:21:05:26 +0000] "GET /vulnerabilities/brute/?username=admin&password=X23RSPrp97&Login=Login HTTP/1.0" 200 4825 "-" "Mozilla/5.0 (Hydra)"
ritik@web-server:~$ _
```

## 6.5 Outcome

The brute force simulation successfully generated DVWA web activity logs. These logs confirmed that Kali Linux was sending repeated login requests to the DVWA brute force module.

---

## 7. SQL Injection Attack Simulation

---

### 7.1 Overview

After completing the brute force simulation, a controlled SQL injection attack was performed against the DVWA SQL Injection module. Kali Linux was used to access DVWA, and the attack was performed through the DVWA web interface.

The purpose of this simulation was to understand how SQL injection works in a vulnerable web application and prepare the generated activity for later detection and analysis in Wazuh.

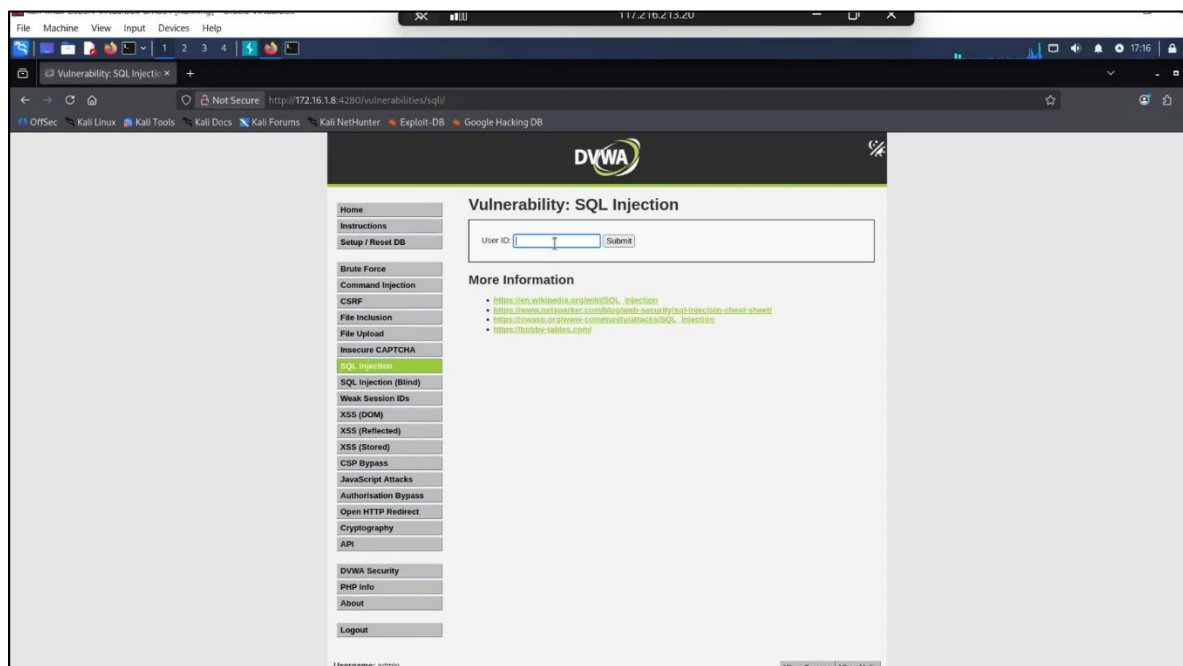
During SQL Injection Attack Simulation, the security level of DVWA was set to low.

### 7.2 Accessing DVWA SQL Injection Module

The SQL Injection module was opened from the DVWA dashboard.

#### URL Used:

<http://172.16.1.8:4280/vulnerabilities/sqli/>



Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

## 7.3 Performing SQL Injection

A basic SQL injection payload was entered into the User ID field.

### Payload Used:

`1' OR '1'='1`

### Explanation:

This payload attempts to manipulate the SQL query logic by making the condition always true. In a vulnerable application, this can return more records than intended.

### Attack Behavior:

The payload was submitted through the DVWA SQL Injection page. This generated a web request to the SQL injection module.

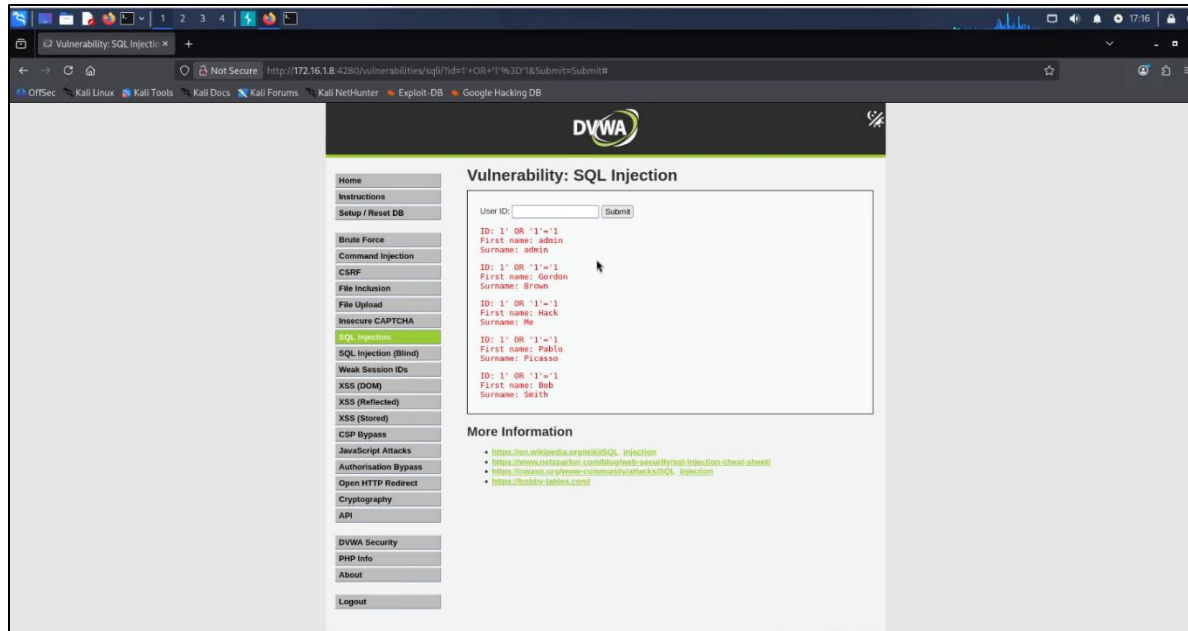


## 7.4 Observing SQL Injection Result in DVWA

After submitting the payload, DVWA returned user information from the backend database. This confirmed that the SQL injection test worked successfully in the vulnerable DVWA environment.

### Observation:

The application returned database records after the injected input was submitted. This showed that the vulnerable SQL query accepted the injected condition.



## 7.5 Outcome

The SQL injection simulation was successfully performed against the DVWA SQL Injection module.

---

## 8. Detection and Analysis Using Wazuh

---

### 8.1 Overview

After performing the brute force and SQL injection simulations against DVWA, the generated activity was analyzed in the Wazuh dashboard. Since Docker log collection and custom local rules were already configured, Wazuh was able to detect DVWA-related web activity from the Ubuntu web server.

The main objective of this section is to analyze how the brute force and SQL injection activities appeared in Wazuh and understand the important alert fields from a SOC analyst perspective.

### 8.2 Brute Force Activity Detection

The brute force simulation generated multiple requests to the DVWA brute force module.

In Wazuh, this activity was detected using the custom local rule:

Rule ID: 100500

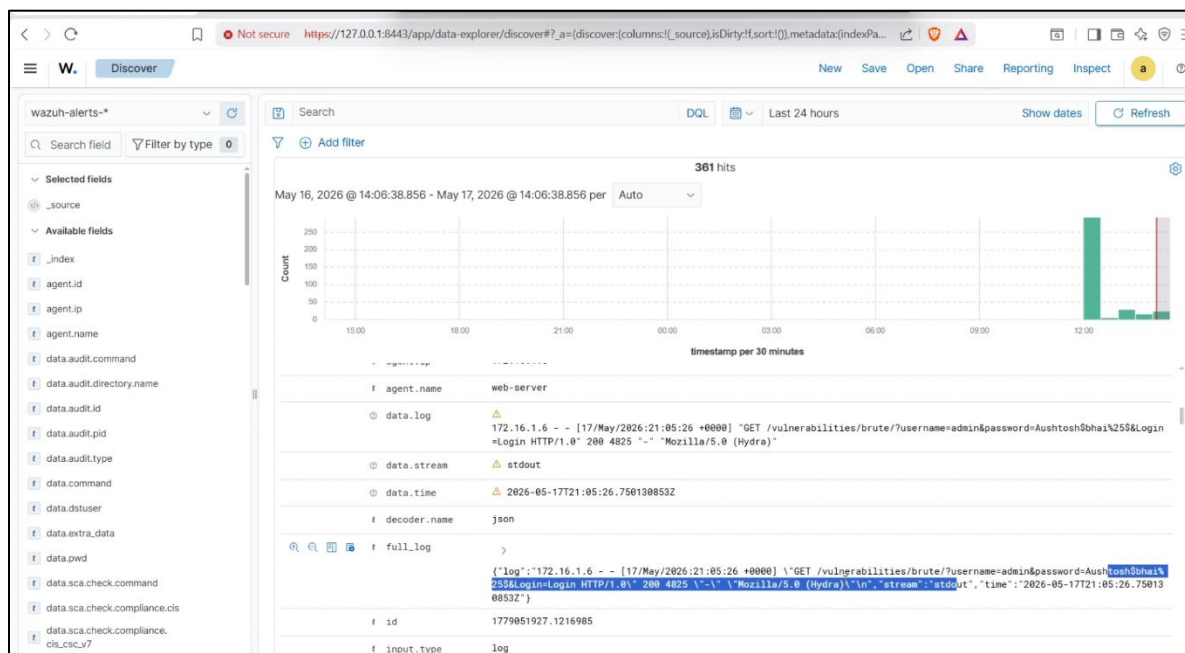
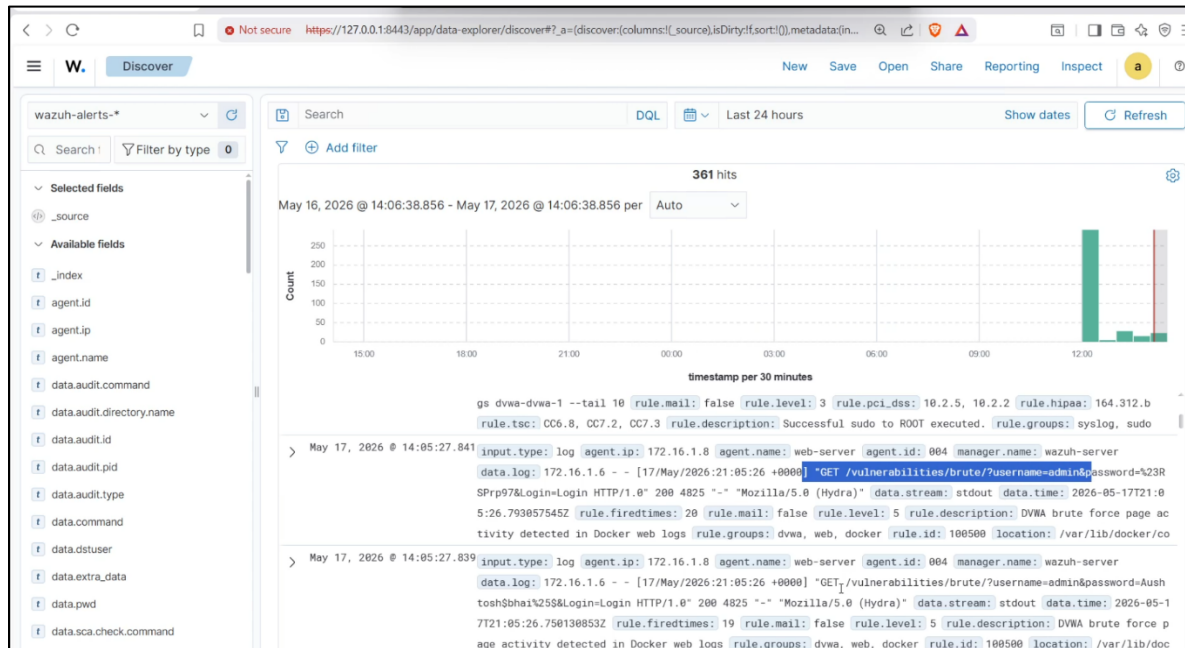
Rule Description: DVWA brute force page activity detected in Docker web logs

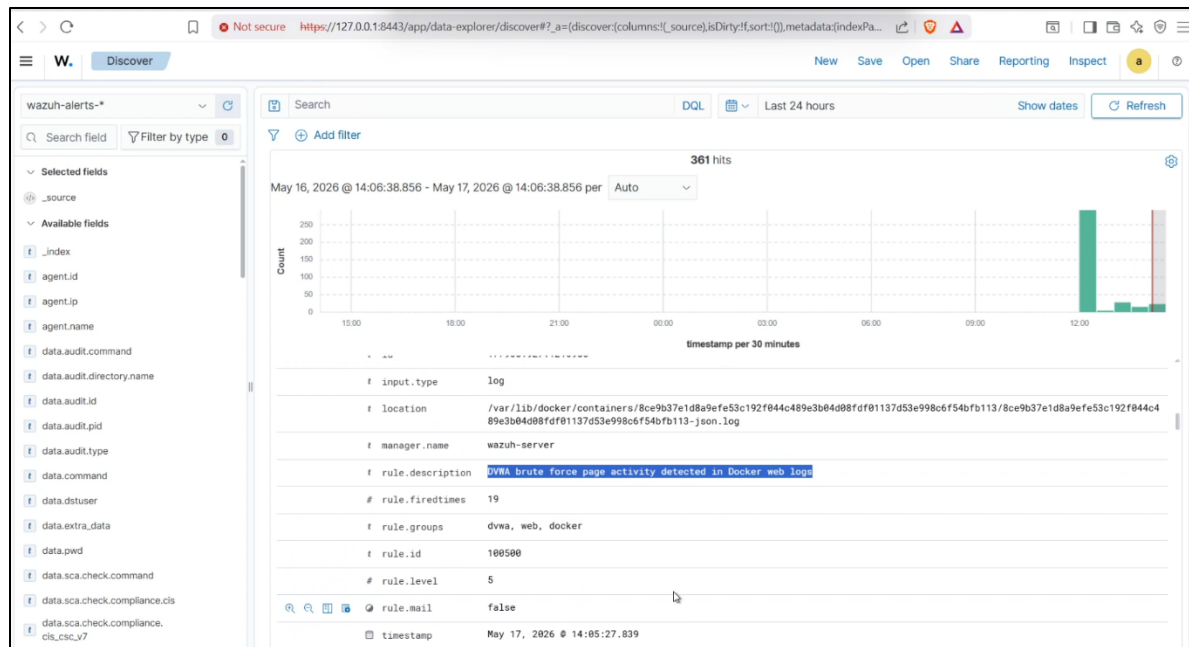
**Key indicators observed:**

- Agent Name: web-server
- Agent IP: 172.16.1.8
- Target Path: /vulnerabilities/brute/
- Log Source: Docker container logs
- Rule Level: 5

## Analysis:

The alert shows that the Ubuntu web server received requests to the DVWA brute force page. Since this activity was generated from Kali Linux and repeated multiple times, it indicates brute force testing against the DVWA login form.





## 8.3 SQL Injection Activity Detection

The SQL injection simulation generated activity against the DVWA SQL Injection module.

In Wazuh, this activity was detected using the custom local rule:

Rule ID: 100501

Rule Description: DVWA SQL injection page activity detected in Docker web logs

### Key indicators observed:

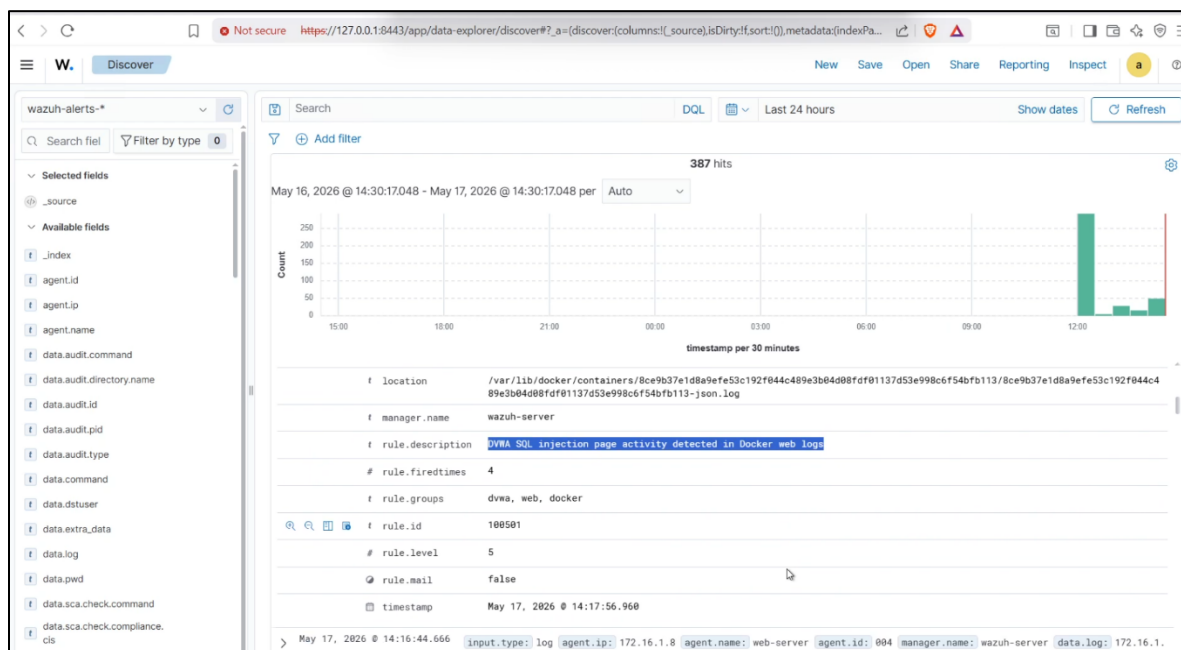
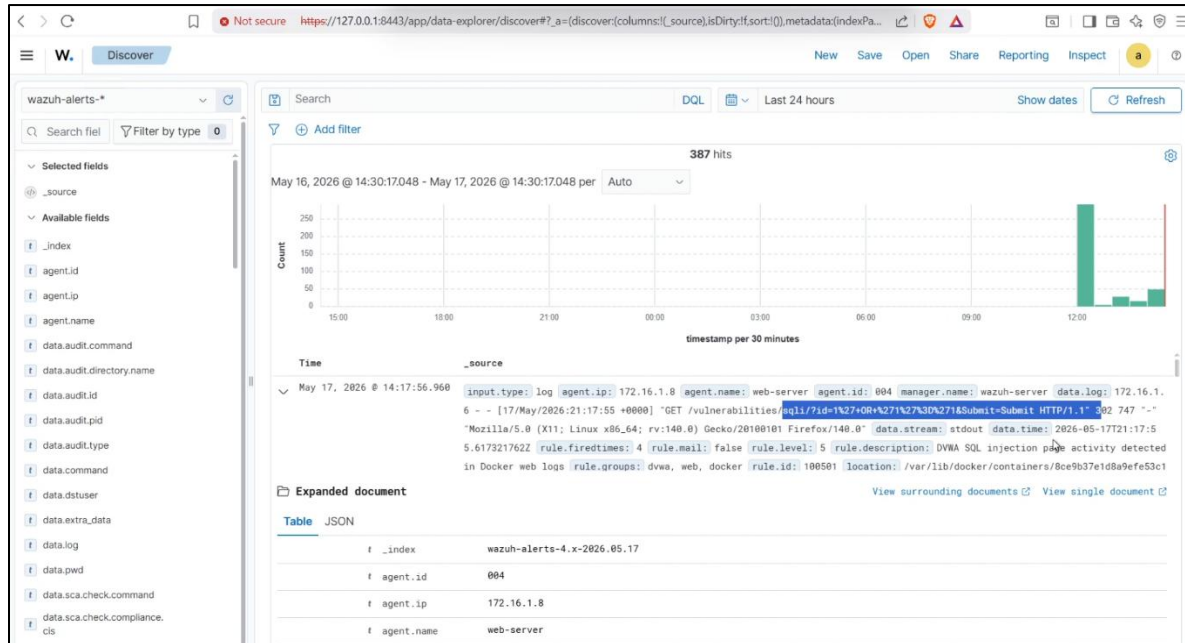
- Agent Name: web-server
- Agent IP: 172.16.1.8
- Target Path: /vulnerabilities/sqli/
- Log Source: Docker container logs
- Rule Level: 5

### Analysis:

The alert shows that the DVWA SQL Injection module was accessed from Kali Linux. The activity matched the custom Wazuh rule created for the /vulnerabilities/sqli/ path.

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>



## 8.4 Outcome

Wazuh successfully detected DVWA brute force and SQL injection activity using Docker log collection and custom local rules.

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>



---

## 9. Conclusion

---

### 9.1 Summary

In this project, DVWA web attack detection was successfully performed in the SOC lab environment using Wazuh SIEM. The Ubuntu web server was connected to Wazuh by installing the Wazuh agent, and Docker log collection was configured because DVWA was running inside Docker containers.

Custom Wazuh local rules were created to detect DVWA brute force and SQL injection activity. After performing controlled attacks from Kali Linux, the generated Docker logs were collected by Wazuh and displayed as alerts in the Wazuh dashboard.

This project demonstrated how containerized web application activity can be monitored and analyzed from a SOC analyst perspective.

### 9.2 Key Achievements

- ✓ Installed and verified the Wazuh agent on the Ubuntu web server
- ✓ Configured Docker container log collection in Wazuh
- ✓ Enabled Docker listener for container-related monitoring
- ✓ Created custom Wazuh rules for DVWA attack detection
- ✓ Performed brute force attack simulation against DVWA
- ✓ Performed SQL injection attack simulation against DVWA
- ✓ Verified DVWA-related alerts in the Wazuh dashboard
- ✓ Analyzed important alert fields such as agent name, rule ID, rule description, and log source

### 9.3 Practical Learning Outcomes

This project provided hands-on experience with:

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

## </Hacker4Help>

- Wazuh agent configuration on a Linux web server
- Docker container log collection
- Web application attack simulation
- Custom Wazuh local rule creation
- Brute force detection using web logs
- SQL injection activity detection
- SIEM alert investigation
- Understanding how containerized application logs can be used in SOC monitoring

### 9.4 MITRE ATT&CK Mapping

| Attack Activity                 | MITRE Tactic                  | MITRE Technique                           |
|---------------------------------|-------------------------------|-------------------------------------------|
| Brute force attempt             | Credential Access             | T1110 – Brute Force                       |
| Password guessing behavior      | Credential Access             | T1110.001 – Password Guessing             |
| SQL injection                   | Initial Access / Exploitation | T1190 – Exploit Public-Facing Application |
| Web application attack activity | Initial Access                | T1190 – Exploit Public-Facing Application |

### 9.5 Real-World Relevance

This lab represents a small version of a real SOC monitoring workflow. In real environments, web applications may run directly on servers, inside Docker containers, or in cloud-based platforms. SOC teams must make sure that the correct logs are collected and that detection rules are tuned properly.

This project showed that simply deploying a web application is not enough. To detect attacks, the correct log source must be collected, parsed, and matched with detection rules. In this case, Docker logs were the key source for identifying DVWA attack activity.

### 9.6 Future Work

Future improvements for this lab may include:

Ritik Patel

SOC - Threat Research intern at </Hacker4Help>

</Hacker4Help>

- Creating more advanced Wazuh rules for actual payload patterns
- Adding MITRE mapping directly inside custom Wazuh rules
- Detecting XSS, command injection, and file upload attacks
- Creating frequency-based brute force detection rules
- Deploying additional PHP web applications for vulnerability testing
- Integrating ModSecurity or a web application firewall
- Comparing Docker logs, Apache logs, and WAF logs for better detection